Focus — la shell moderna: strumenti che valgono l'installazione

> **Materiale facoltativo.** Non è materia d'esame, non si dà per svolto e non serve per nessun esercizio del corso. È per chi finisce prima e si chiede se si possa lavorare meglio: la risposta è sì.

I comandi che hai imparato in `L02_shell.md` hanno dai trenta ai cinquant'anni, e sono installati **su qualunque macchina Unix esistente**. È il loro punto di forza: un comando `grep` scritto oggi funziona su un server a cui ti collegherai fra tre anni, senza chiedere niente a nessuno.

Negli ultimi dieci anni ne sono nate riscritture moderne: stessa funzione, output più leggibile, comportamenti sensati per default. Non sostituiscono i classici — li affiancano.

> **Una regola, prima di cominciare.** Negli script che condividi, nelle soluzioni che consegni e all'esame si usano **solo i comandi classici**. Gli strumenti di questa pagina non sono installati sulle macchine degli altri, e un comando che gira solo sul tuo computer non è un risultato riproducibile.

Come si installano sulla VM

Sulla macchina virtuale del corso non hai i permessi di amministratore, quindi `apt install` non è un'opzione. Ma c'è Anaconda, e `conda` installa nella tua home senza bisogno di permessi:

```
$ conda install -c conda-forge ripgrep fd-find bat
```

Un pacchetto per volta, o tutti insieme come sopra. Se `conda` è lento, `mamba` è lo stesso installatore riscritto e fa la stessa cosa in un decimo del tempo:

```
$ mamba install -c conda-forge ripgrep fd-find bat dust glances glow
```

Tutti gli strumenti di questa pagina sono su **conda-forge**, quindi la procedura è sempre questa. Sul tuo computer, se preferisci: `brew install` su macOS, `apt`/`dnf` su Linux, `winget` o `scoop` su Windows.

`rg` — cercare dentro i file

Sostituisce `grep`. Il nome del pacchetto è `ripgrep`, il comando è `rg`.

```
$ grep -r -i "milano" data/          # il classico
$ rg -i milano data/                 # lo stesso, con rg
```

Cosa cambia:

- **è ricorsivo per default** — niente `-r`, cerca da qui in giù;
- **salta quello che non serve** — file binari e, nei progetti con git, quello che è in `.gitignore`;
- **colora** il testo trovato e raggruppa i risultati per file;

- è molto più veloce su alberi grandi, il che si nota su una cartella di dati.

```
$ rg -c milano data/                # conta le occorrenze per file
$ rg -t csv milano                  # cerca solo nei .csv
$ rg -l milano                      # solo i nomi dei file, non le righe
```

`fd` — cercare i file

Sostituisce `find`. Il pacchetto è `fd-find`, il comando è `fd`.

```
$ find . -name '*.csv'              # il classico
$ fd csv                            # lo stesso, con fd
```

`fd` cerca per sottostringa, senza bisogno di asterischi e apici, ignora le directory nascoste e colora l'output. La sintassi verbosa di `find` — `-name`, `-type`, gli apici obbligatori — è la prima cosa che si dimentica fra una volta e l'altra, ed è esattamente quella che `fd` non chiede.

```
$ fd -e csv                         # per estensione
$ fd -t d dati                      # solo directory il cui nome contiene "dati"
$ fd . data/ -x wc -l               # esegue un comando su ogni risultato
```

`bat` — guardare un file

Sostituisce `cat`, e in parte `less`.

```
$ cat script/analisi.py             # il classico
$ bat script/analisi.py             # con numeri di riga e sintassi colorata
```

Riconosce il linguaggio dall'estensione e colora di conseguenza: su un `.py` o su un `.csv` la differenza è notevole. Impagina da sé quando il file è lungo, quindi si comporta come `less` senza che tu debba scegliere.

```
$ bat -n dati.csv                   # solo numeri di riga, senza cornice
$ bat -r 10:20 analisi.py           # solo le righe da 10 a 20
```

`dust` — capire chi occupa lo spazio

Sostituisce `du`, che non abbiamo nemmeno visto perché il suo output è illeggibile.

```
$ dust                              # albero delle directory per dimensione
$ dust -n 20 ~/dati                 # le 20 voci più grosse
```

Mostra un grafico a barre ordinato dal più grande: in due secondi sai qual è la cartella che ha riempito il disco. Se preferisci qualcosa di interattivo, `ncdu` fa la stessa cosa navigabile con le frecce.

`glances` — cosa sta girando

Sostituisce `top` e `htop`: quanto processore e quanta memoria stanno usando i programmi aperti.

```
$ glances
```

Serve quando un'analisi sembra bloccata e vuoi sapere se sta lavorando o se ha finito la memoria — situazione che incontrerai il primo giorno in cui aprirai un dataset più grande della RAM.

`glow` — leggere un markdown nel terminale

```
$ glow L02_riepilogo.md
```

Rende il markdown formattato — titoli, tabelle, blocchi di codice — dentro il terminale, senza aprire un browser. Utile proprio per i materiali di questo corso, che sono tutti file `.md`.

Due comodità che non sono comandi

Cambiano come ti muovi, non cosa fai.

`zoxide` ricorda le directory in cui sei stato e ti ci porta con una sottostringa: dopo qualche giorno, `z info` ti porta in `~/Corsi/Informatica/2026_27` senza scrivere il percorso.

```
$ conda install -c conda-forge zoxide
```

`eza` è `ls` con i colori per tipo di file, le dimensioni leggibili e l'albero integrato (`eza --tree`).

Cosa installare, se ne installi solo due

`rg` e `bat`. Il primo perché cercare dentro i file è l'operazione che farai più spesso su un dataset che non conosci; il secondo perché guardare un `.csv` o un `.py` colorato costa zero e ti fa vedere gli errori prima.

E ricordati la regola di apertura: quello che consegni gira con i comandi classici.